# Lab 06 – First WPF Window, Styles, Resources

## Title: Introduction to Windows Presentation Foundation (WPF) and XAML

## Objectives

This was my first step into graphical user interfaces with WPF. The goal was to understand the declarative nature of XAML (eXtensible Application Markup Language), how to structure a window using layout panels, and how to use resources for consistent styling. Moving from console to GUI is a major shift in thinking about user interaction.

## Implementation

### MainWindow.xaml

```
<Window x:Class="SalariiUI.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Sistem Salarii - Gestionare Angajați"
        Height="500"
        Width="600"
        WindowStartupLocation="CenterScreen">

    <Window.Resources>
        <!-- Color resources -->
        <SolidColorBrush x:Key="PrimaryBrush" Color="#2C3E50"/>
        <SolidColorBrush x:Key="AccentBrush" Color="#3498DB"/>
        <SolidColorBrush x:Key="LightBrush" Color="#ECF0F1"/>
```

```
    <!-- Style for all Labels -->
    <Style TargetType="Label">
        <Setter Property="FontFamily" Value="Segoe UI"/>
        <Setter Property="FontSize" Value="14"/>
        <Setter Property="Foreground" Value="{StaticResource PrimaryBrush}"/>
        <Setter Property="Margin" Value="5,2"/>
        <Setter Property="FontWeight" Value="SemiBold"/>
    </Style>

    <!-- Style for all Buttons -->
    <Style TargetType="Button">
        <Setter Property="FontFamily" Value="Segoe UI"/>
        <Setter Property="FontSize" Value="14"/>
        <Setter Property="Background" Value="{StaticResource AccentBrush}"/>
        <Setter Property="Foreground" Value="White"/>
        <Setter Property="Padding" Value="15,8"/>
        <Setter Property="Margin" Value="5,2"/>
        <Setter Property="BorderThickness" Value="0"/>
        <Setter Property="Cursor" Value="Hand"/>
        <Style.Triggers>
            <Trigger Property="IsMouseOver" Value="True">
                <Setter Property="Background" Value="#2980B9"/>
            </Trigger>
            <Trigger Property="IsPressed" Value="True">
                <Setter Property="Background" Value="#1A5276"/>
            </Trigger>
        </Style.Triggers>
    </Style>
</Window.Resources>
```

```
    <StackPanel Background="{StaticResource LightBrush}">
        <!-- Title -->
        <Border Background="{StaticResource PrimaryBrush}" Padding="20,15">
            <TextBlock Text="💰 Sistem de Calcul Salarii"
                       FontSize="24"
                       FontWeight="Bold"
                       Foreground="White"
                       HorizontalAlignment="Center"/>
        </Border>

        <!-- Main content -->
        <StackPanel Margin="30,20" HorizontalAlignment="Center">
            <Label x:Name="lblNume" Content="Nume: Ion Popescu" FontSize="16"/>
            <Label x:Name="lblFunctie" Content="Funcție: Programator" FontSize="16"/>
            <Label x:Name="lblSalariu" Content="Salariu Net: 6,150.00 RON" FontSize="16"
                   Foreground="{StaticResource AccentBrush}" FontWeight="Bold"/>

            <Button Content="Închide" Width="150" Margin="0,20,0,0"
                    HorizontalAlignment="Center"
                    Click="BtnInchide_Click"/>
        </StackPanel>
    </StackPanel>
</Window>
```

## Code-Behind (MainWindow.xaml.cs)

```
using System.Windows;

namespace SalariiUI
{
```

```
  public partial class MainWindow : Window
  {
    public MainWindow()
    {
      InitializeComponent();

      // You could set data here programmatically
      // lblNume.Content = "Nume: " + employee.Name;
    }

    private void BtnInchide_Click(object sender, RoutedEventArgs e)
    {
      this.Close();
    }
  }
}
```

## Why I Used This Approach

**XAML Declarative Syntax**: XAML allows me to define the UI structure in a declarative manner, separate from the code-behind logic. This is cleaner and more maintainable than creating UI elements programmatically.

**Window.Resources**: Resources defined at the window level are available throughout the window. This is where I define reusable brushes and styles.

**Styles**: By defining styles for `Label` and `Button` controls, I can apply consistent formatting to all controls of that type without repeating the same properties. This promotes consistency and reduces code duplication.

**SolidColorBrush Resources**: Defining colors as resources with meaningful keys (PrimaryBrush, AccentBrush) makes theming easier. If I want to change the primary color, I just update it in one place.

**StackPanel**: I used `StackPanel` as the main container because it arranges child elements vertically in a simple, predictable way. For this initial interface, it was sufficient.

**Triggers in Styles**: The button style includes triggers for `IsMouseOver` and `IsPressed`, providing visual feedback to the user. This improves the user experience.

**Name Elements**: Using `x:Name` on labels allows me to access them from code-behind if needed.

## Testing & Results

When I ran the application:

- A window appeared with a professional-looking title in a dark blue header
- Three information labels displayed employee data
- The labels used consistent styling with the defined font and colors
- The button had hover effects (changing color when mouse was over it)
- Clicking the button closed the application

The visual appearance was clean and consistent, demonstrating how WPF's styling system works.

## Reflection

WPF's approach to UI design is very different from Windows Forms. In WPF, the UI is declared in XAML, which is more flexible and powerful. The separation between UI structure (XAML) and behavior (C#) is cleaner. Using resources for styling means I can change the entire look of the application by editing just a few lines, which is incredibly powerful for theming and maintenance. The declarative nature of XAML takes some getting used to, but it's much more expressive once you understand it.